



SCHOOL OF TECHNOLOGY

DATABASE MANAGEMENT SYSTEMS (DBMS) COURSE PROJECT REPORT

DINEDESK: RESTAURANT RESERVATION, REAL-TIME DINING & BILLING MANAGEMENT SYSTEM

SUBMITTED BY:

Student Name: **Sakhunala Sahaja**

Roll Number: 25WU0102237

Course: CS301 - Database Management Systems Laboratory

UNDER THE GUIDANCE OF:

Faculty Name: Dr. Kiran Mayee

Academic Year: 2025 – 2026

WOXSEN UNIVERSITY

1. Executive Summary

Modern culinary establishments face immense logistical bottlenecks when coordinating tableside dining experiences, including table allocation overbooking, fragmented kitchen-line communications, disjointed item-level billing, and data loss during customer draft sessions.

DineDesk is an enterprise-grade, relational database-driven Fine Dining & Lounge Management System engineered to bridge the disconnect between patrons, executive kitchen brigades, and front-of-house staff. Built on a Third Normal Form (3NF) relational architecture using SQLite and FastAPI backend microservices, DineDesk delivers end-to-end operational visibility.

2. Abstract

This project presents the full lifecycle design, relational normalization, implementation, and operational validation of DineDesk. The database architecture is normalized to 3NF/BCNF, comprising 20 interconnected relational tables that manage physical dining areas, seating tables, guest reservations, floor dining sessions, digital menu catalogs, customer orders, Kitchen Display System (KDS) prep tickets, invoices, promotional discounts, and payment settlements.

3. System Architecture and Scope

3.1 Primary Objectives

- Design and normalize a robust relational schema up to 3NF/BCNF enforcing strict entity and referential integrity.
- Develop automated RESTful CRUD microservices for floor seating, menu cataloging, dining sessions, orders, and payment receipts.
- Enforce business rules at the database level: foreign key cascades, unique table constraints, timestamp range queries, and reservation lead-time guarantees.
- Provide an intuitive, responsive frontend with persistent state caching, dynamic filtering, locked table indicators, and kitchen progress visualization.

3.2 Technical Stack

Component	Technology	Purpose
Backend Language	Python 3.10+	Core business logic and microservice routing
Web Framework	FastAPI (Uvicorn)	Asynchronous REST API dispatch

Database Engine	SQLite 3 / PostgreSQL	ACID-compliant relational persistence
ORM Layer	SQLAlchemy 2.0	Declarative schema modeling and query composition
Frontend UI	HTML5, CSS3, ES6 JavaScript	Responsive front-of-house and KDS interfaces

4. Entity Relationship (ER) Diagram

The DineDesk database architecture models 20 core entities connected through foreign-key relationships. Below is the comprehensive Chen-Style Entity-Relationship Diagram highlighting physical dining areas, floor tables, customer reservations, active dining sessions, menu categories, customer orders, kitchen display tickets, invoices, and payment receipts:

5. Relational Schema and Normalization

5.1 Relational Schema Representation

```

1. BRANCH (branch_id [PK], name, phone, address, created_at, updated_at)
2. DINING_AREA (area_id [PK], branch_id [FK->BRANCH.branch_id], name, created_at, updated_at)
3. TABLE_ENTITY (table_id [PK], area_id [FK->DINING_AREA.area_id], table_number [UQ], capacity, is_active, created_at, updated_at)
4. ROLE (role_id [PK], name [UQ], description)
5. USER_ACCOUNT (user_id [PK], role_id [FK->ROLE.role_id], branch_id [FK->BRANCH.branch_id], email [UQ], first_name, last_name, password_hash, created_at)
6. CUSTOMER (customer_id [PK], first_name, last_name, phone [UQ], email [UQ], created_at, updated_at)
7. RESERVATION (reservation_id [PK], table_id [FK->TABLE_ENTITY.table_id], customer_id [FK->CUSTOMER.customer_id], start_time, end_time, guest_count, status, special_requests, created_at, updated_at)
8. DINING_SESSION (session_id [PK], table_id [FK->TABLE_ENTITY.table_id], customer_id [FK->CUSTOMER.customer_id], reservation_id [FK->RESERVATION.reservation_id], guest_count, status, start_time, end_time, created_at, updated_at)
9. MENU_CATEGORY (category_id [PK], name [UQ], description)
10. MENU_ITEM (item_id [PK], category_id [FK->MENU_CATEGORY.category_id], name, description, current_price, is_active, image_url, created_at, updated_at)
11. CUSTOMER_ORDER (order_id [PK], session_id [FK->DINING_SESSION.session_id], user_id [FK->USER_ACCOUNT.user_id], status, created_at, updated_at)
12. ORDER_ITEM (order_item_id [PK], order_id [FK->CUSTOMER_ORDER.order_id], item_id

```

```
[FK->MENU_ITEM.item_id], quantity, unit_price, special_requests, created_at,
updated_at)
13. ORDER_STATUS_HISTORY (history_id [PK], order_id [FK->CUSTOMER_ORDER.order_id],
status, changed_at)
14. KITCHEN_TICKET (ticket_id [PK], order_id [FK->CUSTOMER_ORDER.order_id], status,
prep_start_time, ready_time, created_at, updated_at)
15. KITCHEN_TICKET_ITEM (ticket_item_id [PK], ticket_id [FK-
>KITCHEN_TICKET.ticket_id], order_item_id [FK->ORDER_ITEM.order_item_id], status)
16. BILL (bill_id [PK], session_id [FK->DINING_SESSION.session_id], subtotal,
tax_amount, total_amount, status, created_at, updated_at)
17. BILL_ITEM (bill_item_id [PK], bill_id [FK->BILL.bill_id], order_item_id [FK-
>ORDER_ITEM.order_item_id], item_name_snapshot, quantity, total_price)
18. DISCOUNT (discount_id [PK], code [UQ], discount_type, percentage_off,
fixed_amount_off, is_active)
19. DISCOUNT_APPLICATION (discount_app_id [PK], bill_id [FK->BILL.bill_id],
discount_id [FK->DISCOUNT.discount_id], authorized_by [FK->USER_ACCOUNT.user_id],
applied_amount)
20. PAYMENT (payment_id [PK], bill_id [FK->BILL.bill_id], amount, payment_method,
status, transaction_ref [UQ], created_at, updated_at)
```

5.2 Normalization Proofs

- First Normal Form (1NF): All attributes across all 20 relations are strictly atomic (single-valued). Multi-valued elements such as dietary tags or special chef requests are decoupled into child tables or structured string attributes.
- Second Normal Form (2NF): The schema satisfies 1NF and exhibits no partial functional dependencies. Non-key attributes in associative entities (such as quantity and unit_price in ORDER_ITEM) depend fully on the complete primary key.
- Third Normal Form (3NF): The schema satisfies 2NF and has zero transitive dependencies. Historical prices and invoice line totals are snapshotted into immutable BILL_ITEM records to protect historical billing records against future menu price updates.

6. Data Dictionary and Schema Specifications

6.1 Table Specification: TABLE_ENTITY

Field Name	Data Type	Constraints	Description
id	VARCHAR(36)	PRIMARY KEY	Unique UUID string identifier

area_id	VARCHAR(36)	FOREIGN KEY	References dining_area(id) ON DELETE SET NULL
table_number	VARCHAR(10)	NOT NULL, UNIQUE	Physical floor table code (e.g. T01, R03, F01)
capacity	INTEGER	NOT NULL, CHECK(>0)	Maximum seating guest capacity
is_active	BOOLEAN	DEFAULT 1	Active status indicator flag

6.2 Table Specification: RESERVATION

Field Name	Data Type	Constraints	Description
id	VARCHAR(36)	PRIMARY KEY	Unique reservation booking identifier
table_id	VARCHAR(36)	FOREIGN KEY	References table_entity(id) ON DELETE CASCADE
customer_id	VARCHAR(36)	FOREIGN KEY	References customer(id) ON DELETE CASCADE
start_time	DATETIME	NOT NULL	Booking window commencement timestamp
end_time	DATETIME	NOT NULL	Booking window conclusion timestamp
guest_count	INTEGER	NOT NULL, CHECK(>0)	Number of booked dining guests
status	VARCHAR(20)	DEFAULT 'CONFIRMED'	Booking status (PENDING, CONFIRMED, CANCELLED)

6.3 Table Specification: KITCHEN_TICKET & KITCHEN_TICKET_ITEM

Field Name	Data Type	Constraints	Description
id	VARCHAR(36)	PRIMARY KEY	Unique kitchen ticket identifier
order_id	VARCHAR(36)	FOREIGN KEY	References customer_order(id) ON DELETE CASCADE
status	VARCHAR(20)	DEFAULT 'PENDING'	Ticket status (PENDING, IN_PROGRESS, READY)
prep_start_time	DATETIME	NULLABLE	Timestamp chef commenced cooking
ready_time	DATETIME	NULLABLE	Timestamp all dishes completed prep

7. SQL Implementation (DDL and DML)

7.1 Data Definition Language (DDL)

```
CREATE TABLE dining_area (  
  id VARCHAR(36) PRIMARY KEY,  
  name VARCHAR(50) NOT NULL UNIQUE,  
  description TEXT,  
  is_active BOOLEAN DEFAULT 1  
);  
  
CREATE TABLE table_entity (  
  id VARCHAR(36) PRIMARY KEY,  
  area_id VARCHAR(36) REFERENCES dining_area(id) ON DELETE SET NULL,  
  table_number VARCHAR(10) NOT NULL UNIQUE,  
  capacity INTEGER NOT NULL CHECK (capacity > 0),  
  is_active BOOLEAN DEFAULT 1,  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE reservation (  
    id VARCHAR(36) PRIMARY KEY,  
    table_id VARCHAR(36) NOT NULL REFERENCES table_entity(id) ON DELETE CASCADE,  
    customer_id VARCHAR(36) NOT NULL REFERENCES customer(id) ON DELETE CASCADE,  
    start_time DATETIME NOT NULL,  
    end_time DATETIME NOT NULL,  
    guest_count INTEGER NOT NULL CHECK (guest_count > 0),  
    status VARCHAR(20) DEFAULT 'CONFIRMED',  
    special_notes TEXT,  
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

7.2 Data Manipulation Language (DML)

```
INSERT INTO dining_area (id, name, description) VALUES  
(  
    'AREA-01', 'Romantic Terrace (Rooftop)', 'Sunset ambiance with panoramic city skyline views',  
    'AREA-02', 'Main Dining Hall', 'Opulent crystal chandeliers and live classical lounge acoustics',  
    'AREA-03', 'Family & Kids Lounge', 'Spacious plush seating with interactive dessert stations',  
    'AREA-04', 'Chef's Private Dining', 'Exclusive private suite featuring sommelier pairings'  
);  
  
INSERT INTO table_entity (id, area_id, table_number, capacity) VALUES  
(  
    'TBL-R01', 'AREA-01', 'R01', 2),  
    ('TBL-R02', 'AREA-01', 'R02', 2),  
    ('TBL-R03', 'AREA-01', 'R03', 4),  
    ('TBL-T01', 'AREA-02', 'T01', 4),  
    ('TBL-T02', 'AREA-02', 'T02', 4),  
    ('TBL-F01', 'AREA-03', 'F01', 6),  
    ('TBL-P01', 'AREA-04', 'P01', 8);
```

8. Database Queries and Verification Results

8.1 Real-Time Kitchen Display Ticket Inspection

Query execution output demonstrating live ticket items, quantities, ticket prep statuses, and chef special requests:

Ticket Ref	Table	Ticket Status	Item Name	Qty	Item Status	Chef Notes
------------	-------	---------------	-----------	-----	-------------	------------

6eb2cf3a	T01	PENDING	Baked New York Raspberry Cheesecake	1	PENDING	None
6eb2cf3a	T01	PENDING	Cold Brew Iced Vanilla Latte	1	PENDING	None
593a572f	F01	PENDING	Crispy Corn & Jalapeno Croquettes	1	PENDING	None
593a572f	F01	PENDING	Dal Makhani Bukhara	1	PENDING	None
6349d9db	R03	IN_PROGRESS	Chicken Alfredo Fettuccine	1	PREPARING	None
6349d9db	R03	IN_PROGRESS	Classic Margherita Pizza	1	PREPARING	None
dd12373f	F01	IN_PROGRESS	Classic Margherita Pizza	1	PREPARING	Extra mild sauce & small triangles for kid
dd12373f	F01	IN_PROGRESS	Butter Garlic Naan	3	PREPARING	Well done, crispy edges
7985c3ce	T03	READY	Dum Murgh Awadhi Biryani	2	COMPLETED	Authentic Dum prep with raita
7985c3ce	T03	READY	Sizzling Chocolate Walnut Brownie	2	COMPLETED	Extra vanilla scoop

9. Conclusion

The DineDesk restaurant platform successfully demonstrates a robust 3NF relational database architecture designed for high-concurrency fine dining operations. By linking front-of-house table seating with live Kitchen Display System (KDS) streams and automated invoice settlements, DineDesk eliminates communication bottlenecks and guarantees complete transactional integrity.